Oracle Database 19c RAC 과정

RAC 스토리지 관리

리두·UNDO·TEMP·데이터 파일의 공유와 분리

제10장

학습 목표

- 리두 스레드와 멤버를 구분하고 인스턴스와 대응시킨다
- 스레드를 지정해 리두 그룹을 추가·삭제한다
- 인스턴스별 UNDO 배정과 ORA-30013·30012를 구분한다
- TEMP의 인스턴스별 확보 구조를 설명한다
- 공유와 분리의 기준, 노드 추가 때 준비물을 말한다

Agenda

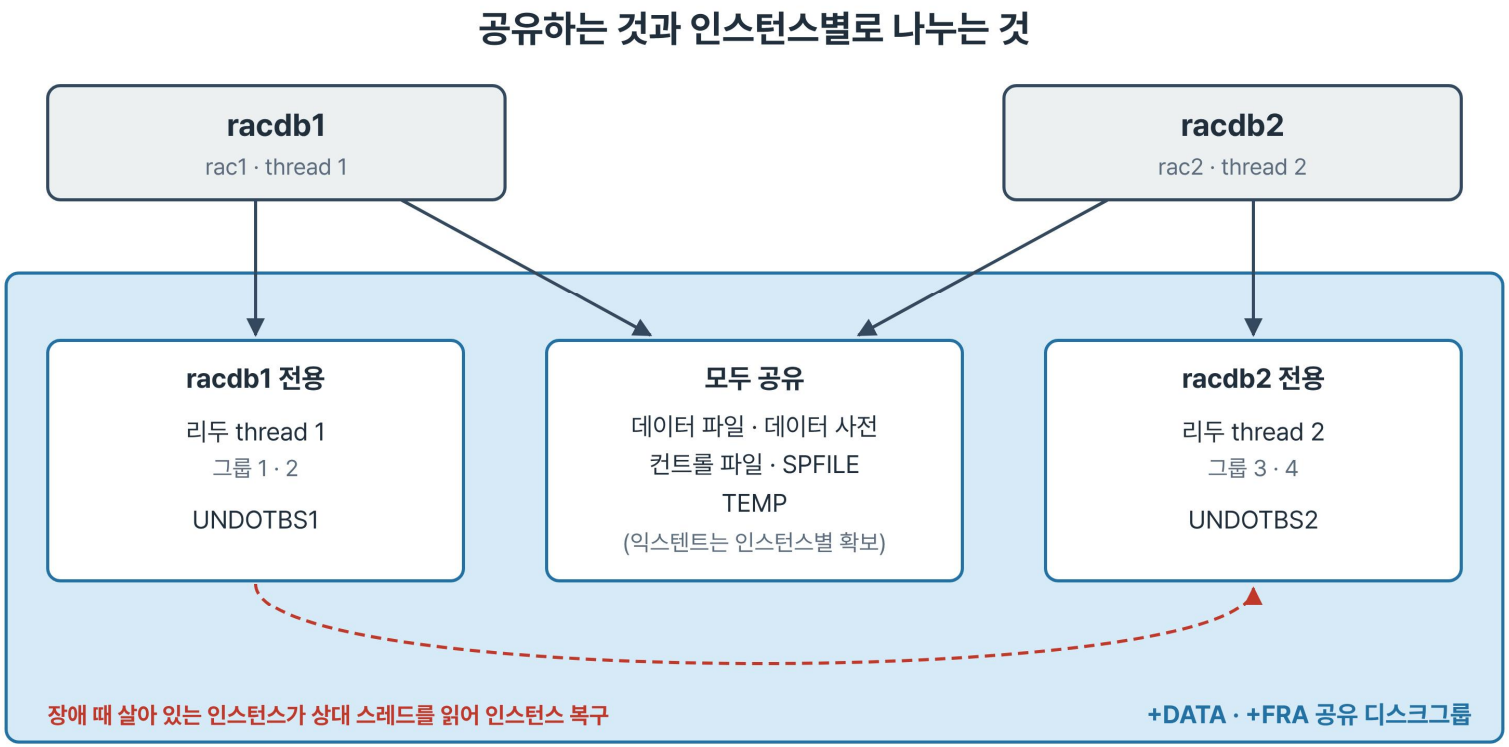
- 10.2 공유하는 것과 나누는 것
- 10.3 온라인 리두와 스레드
- 10.4 UNDO 테이블스페이스
- 10.5 임시 테이블스페이스 / 10.6 데이터 파일
- 10.7 노드 추가와 스토리지 점검

10.2

공유하는 것과 나누는 것

- 자원별 배치
- 나누는 기준

공유와 분리의 전체 그림



나눈다 — 순차 쓰기(리두), 배타적 배정(UNDO)

공유한다 — 구조 정보, 데이터, 익스텐트 단위로 할당하는 TEMP

구조 정보는 공유하고 리두와 UNDO만 나눈다

나누는 기준

동시에 써도 되는가

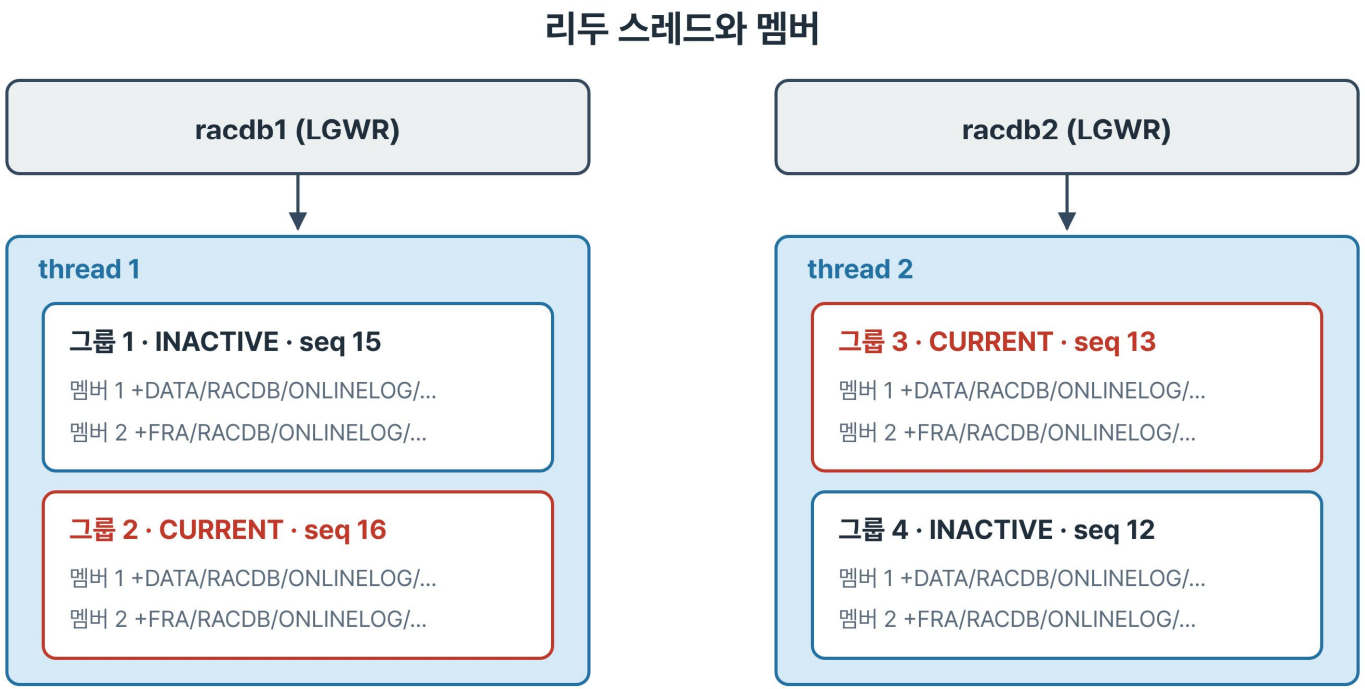
- 리두: 순차 쓰기, 한 파일 끝에 둘이 이어 붙일 수 없음
- UNDO: 롤백 세그먼트를 배타적으로 배정
- TEMP: 익스텐트 단위 할당이라 경합이 작음
- 구조 정보는 언제나 공유, 나뉘는 것은 사용

10.3

온라인 리두와 스레드

- 스레드와 인스턴스
- 스레드와 멤버
- 그룹 추가와 삭제

리두 스레드와 멤버



스레드 = 누가 쓰는가 (RAC 에서 생긴 축) · 스레드마다 CURRENT 하나, SEQUENCE# 도 따로

멤버 = 몇 벌 두는가 (다중화, 단일 인스턴스에도 있음)

4 그룹 × 2 멤버 → V$LOGFILE 조인 결과 8행

4개 그룹이 두 스레드로, 그룹마다 멤버 2개

V$LOG로 본 스레드

CURRENT가 두 개인 것이 정상

- 그룹 1·2는 thread 1, 3·4는 thread 2
- 스레드마다 CURRENT 하나
- SEQUENCE#는 스레드 안에서만 증가
- 복구 때는 SCN 순서로 스레드를 병합

V$THREAD 읽기

스레드를 누가 쓰는가

- STATUS OPEN: 지금 쓰는 인스턴스가 있음
- ENABLED PUBLIC: 어느 인스턴스든 사용 가능
- 스레드 지정은 SPFILE의 thread 파라미터
- 오래 CLOSED인 PUBLIC 스레드는 정리 대상

스레드와 멤버

두 축을 구분한다

- 스레드: 누가 쓰는가, RAC에서 생긴 축
- 멤버: 몇 벌 두는가, 다중화
- +DATA와 +FRA에 멤버 하나씩
- 스레드마다 그룹 크기는 같게

리두 그룹 추가

thread 절을 빠뜨리지 않는다

- add logfile thread 1 group 5 ('+DATA','+FRA')
- thread 절 생략 시 실행한 인스턴스의 스레드로
- 새 그룹 STATUS는 UNUSED
- rac2에서도 곧바로 보이지만 쓰지는 않음

ORA-01623

지금 쓰는 그룹은 지울 수 없다

- log 2 is current log for instance racdb1 (thread 1)
- 메시지에 인스턴스와 스레드가 함께 나옴
- ORA-00312 두 줄은 멤버 파일 경로
- 스위치는 그 스레드의 인스턴스에서

리두 크기를 바꾸는 순서

리두 크기를 바꾸는 순서

① 새 크기 그룹을 스레드마다 먼저 추가

```
add logfile thread 1 group 11 (…) size 500M
```

② 그 스레드의 인스턴스에서 로그 스위치

```
alter system switch logfile;
```

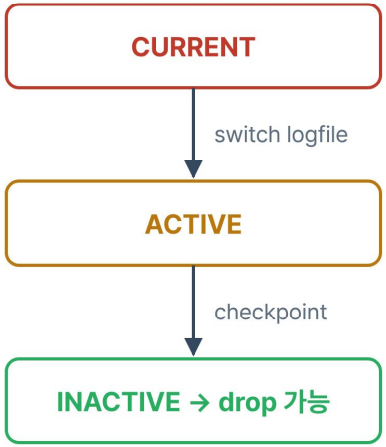
③ 체크포인트 후 INACTIVE 확인

```
alter system checkpoint; select … from v$log;
```

④ 옛 그룹 삭제 (스레드마다 2개 이상 남김)

```
alter database drop logfile group 1;
```

그룹 상태 변화



스위치는 그 스레드를 쓰는 인스턴스에서

thread 1 → rac1 의 racdb1, thread 2 → rac2 의 racdb2. racdb2 에서 스위치해도 thread 1 CURRENT 는 그대로

CURRENT 를 지우면 ORA-01623, 옛 그룹을 먼저 지워 2개 미만이 되면 거부

새 그룹 먼저, 스위치는 해당 인스턴스에서

리두 크기 변경 작업

야간 작업으로 계획한다

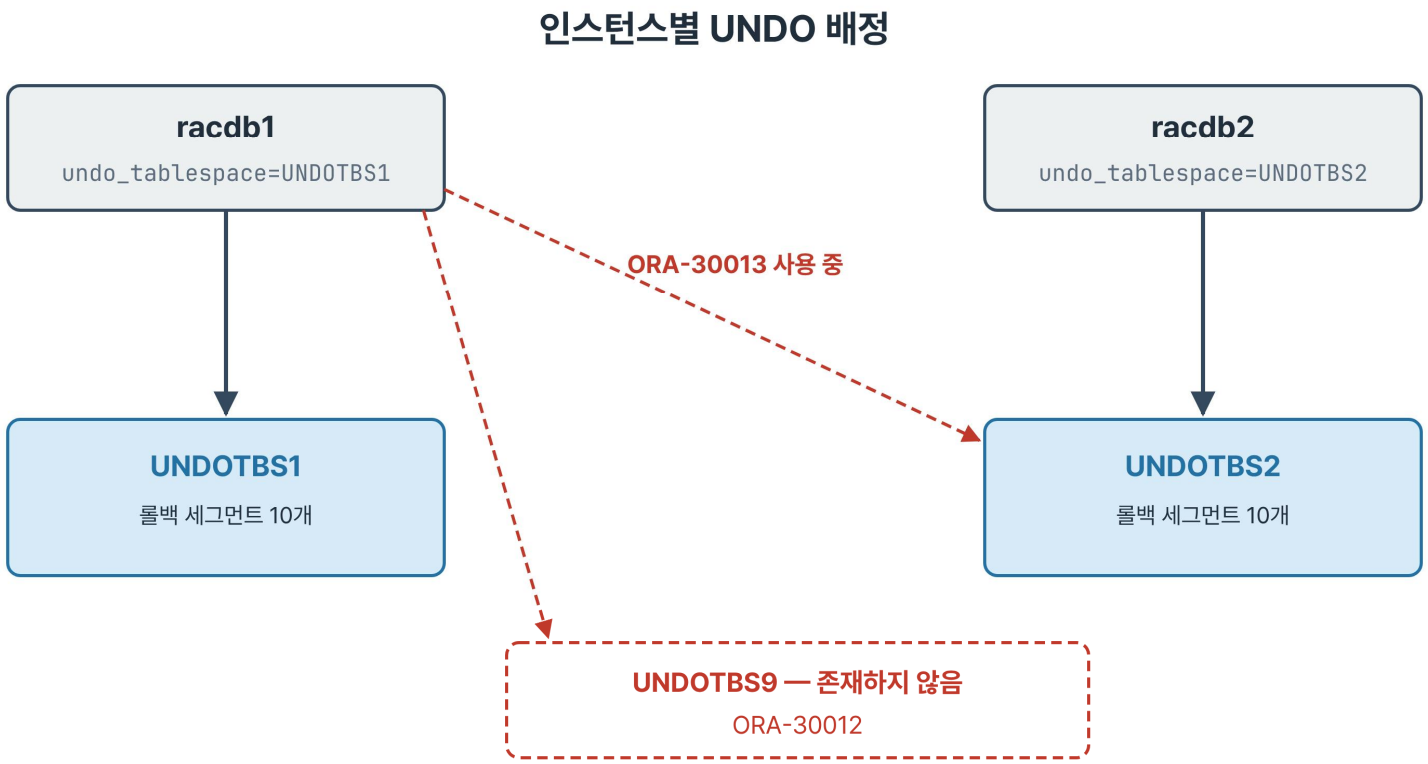
- 스위치·체크포인트 반복으로 I/O 부하 발생
- rac1·rac2 세션을 각각 열고 진행
- 중단하면 옛 크기와 새 크기가 섞임
- 끝나면 V$LOG로 스레드별 그룹 확인

10.4

UNDO 테이블스페이스

- 인스턴스별 배정
- 사용 통계
- ORA-30013과 ORA-30012

인스턴스별 UNDO 배정



같게 둔다 undo_management=AUTO, undo_retention=900

다르게 둔다 undo_tablespace (SPFILE 의 sid 접두어 항목)

남의 것은 ORA-30013, 없는 것은 ORA-30012

배정 확인

같은 것과 다른 것

- gv$parameter의 undo_tablespace: UNDOTBS1, UNDOTBS2
- SPFILE의 sid 접두어 항목으로 지정
- undo_management, undo_retention은 같은 값
- 보존 기간이 다르면 노드마다 ORA-01555 시점이 달라짐

롤백 세그먼트와 CDB 조회

나누는 이유와 조회 주의점

- UNDO마다 롤백 세그먼트 10개
- 트랜잭션은 자기 UNDO의 세그먼트만 배정
- V$TABLESPACE·V$DATAFILE 조인에 con_id 포함
- 인스턴스 축과 컨테이너 축이 곱해진다

크기와 사용 통계

GV$UNDOSTAT으로 두 인스턴스를 함께

- UNDOTBS1 350MB, UNDOTBS2 25MB
- 한쪽만 계속 커지면 부하 편중 신호
- 10분 구간별 UNDOBLKS, MAXQUERYLEN
- MAXQUERYLEN이 retention을 넘으면 ORA-01555 위험

ORA-30013과 ORA-30012

마지막 줄이 원인이다

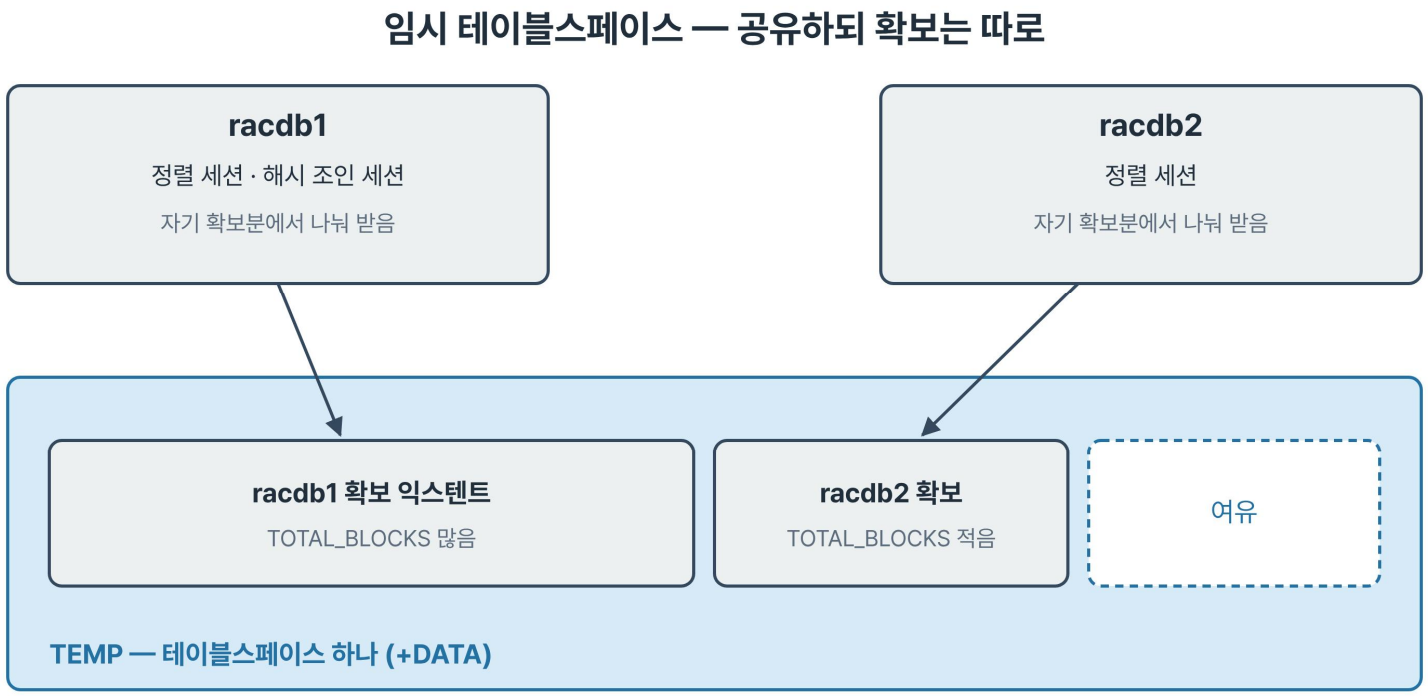
- ORA-02097 다음 줄이 실제 원인
- 30013: 다른 인스턴스가 사용 중
- 30012: 없거나 UNDO 유형이 아님
- scope=memory로 시험하면 SPFILE에 남지 않음

10.5

임시 테이블스페이스

- 하나를 공유
- 인스턴스별 확보량

TEMP는 공유, 확보는 따로



GV$SORT_SEGMENT 인스턴스별 확보량 (TOTAL_BLOCKS · USED_BLOCKS)

GV$TEMPSEG_USAGE 지금 임시 공간을 쓰는 세션

ORA-01652 → 전체 여유와 인스턴스별 확보량을 함께 본다

GV$SORT_SEGMENT에 인스턴스별 행이 나온다

ORA-01652 진단

전체 여유와 인스턴스별 확보량

- 테이블스페이스 전체 크기와 자동 확장
- GV$SORT_SEGMENT의 인스턴스별 TOTAL_BLOCKS
- 한쪽이 대부분을 확보하면 다른 쪽이 부족
- 단일 인스턴스에는 없는 상황

10.6

데이터 파일

- 완전 공유
- 파일 위치

데이터 파일은 완전히 공유

어느 노드에서 만들든 바로 보인다

- rac2에서 만든 TS_CH13이 rac1에서 즉시 ONLINE
- 데이터 사전과 컨트롤 파일을 공유
- 두 인스턴스가 같은 테이블에 동시 입력 가능
- 같은 블록의 조정은 Cache Fusion 몫

파일 위치는 디스크그룹으로

로컬 경로는 다른 노드가 못 연다

- datafile '+DATA' 처럼 디스크그룹 지정
- 로컬 경로 파일은 다른 노드에서 ORA-01157
- OMF가 이름을 짓게 둔다
- 8장의 마이그레이션 사례와 같은 원리

10.7

노드 추가와 스토리지 점검

- 새 인스턴스의 준비물
- 일상 점검

노드 추가 때 준비물

리두·스레드·UNDO 세 가지

- 새 스레드용 리두 그룹 최소 2개
- enable public thread 3
- UNDOTBS3 생성 후 sid 접두어로 지정
- TEMP와 데이터 파일은 그대로

일상 점검 항목

RAC 특유의 두 항목

- 매일: 스레드별 CURRENT, UNDO 크기 편차
- 주간: 스레드별 그룹 수, TEMP 확보량 균형
- 변경 후: 양 노드에서 같은 구성
- UNDO와 TEMP 편차가 단일 인스턴스에 없는 항목

부하 편중 지표

리두로 쓰기 부하를 비교한다

- v$log_history를 스레드별 건수로 집계
- STATUS 열은 없음, 넣으면 ORA-00904
- gv$sysstat의 redo size로 변경량 비교
- 기동 시각이 다르면 시간당 값으로 환산

요약

- 데이터 파일·컨트롤 파일·TEMP는 공유, 리두와 UNDO는 인스턴스별
- 스레드는 누가 쓰는가, 멤버는 몇 벌 두는가
- 그룹 추가는 thread 절, 스위치는 그 스레드의 인스턴스에서
- 남의 UNDO는 ORA-30013, 없는 UNDO는 ORA-30012
- TEMP 부족은 인스턴스별 확보량까지 확인
- 노드 추가: 리두 그룹, 스레드 활성화, UNDO

퀴즈

- 1. 리두·UNDO는 나누고 TEMP는 공유하는 기준은?
- 2. CURRENT가 둘이고 SEQUENCE#가 다른 이유는?
- 3. 그룹 4개 멤버 2개인데 8행인 이유는?
- 4. ORA-01623 때 어느 노드에서 무엇을 하는가?
- 5. ORA-30013과 30012의 차이는?
- 6. TEMP 여유가 있는데 ORA-01652가 나면?

실습 개요

- 01 스레드별 리두 확인 — 스레드와 멤버
- 02 ★ 리두 그룹 추가 — rac2 확인, ORA-01623, 정리
- 03 UNDO 구성 — 배정과 CDB 조인 주의
- 04 오류 재현 — ORA-30013·30012
- 파일: RAC/13장/13장_실습_NN_*.txt